CHAPTER 5

Thursday: Class Definitions

As you know, writing object-oriented programs means spending a good chunk of your time defining classes. In Java and C#, defining a class is like making a deal between you and the compiler. You say, "Here's how my objects are supposed to behave," and the compiler replies, "Okay, they will." Nothing really happens until you create an object of that class and then call that object's methods.

In Ruby, class definitions are different. When you use the `class` keyword, you aren't just dictating how objects will behave in the future. On the contrary, you're actually *running code*.

If you buy into this notion—that a Ruby class definition is actually regular code that runs—you'll be able to cast some powerful spells. Two such spells that you'll learn about in this chapter are *Class Macros (117)* (methods that modify classes) and *Around Aliases (134)* (methods that wrap additional code around other methods). To help you make the most of these spells, this chapter also describes *singleton classes*, one of Ruby's most elegant features. Singleton classes are an advanced topic, so understanding them will win you bragging rights among Ruby experts.

This chapter also comes with a couple of public service announcements. First, keep in mind that a class is just a souped-up module, so anything you learn about classes also applies to modules. Although I won't repeat this PSA in every section of this chapter, remember that whenever you read about a "class definition," you can also think to yourself "module definition." Second, be prepared: this is probably the most advanced chapter in the entire book. Read through it, and you will be able to walk the darkest corners of the Ruby object model.